PROGRAMMING LANGUAGES

# C / C++

A systems-programmer's guide to the memory model, pointers, RAII, templates, the STL, and the undefined-behavior pitfalls that separate working code from code that only looks like it works.

c · c++ · memory-management · pointers · raii · stl · smart-pointers

Study Pack · Comprehensive reference & interview prep

CONTENTS


1. What C and C++ Are, and Why They Still Matter
2. Setup and Getting Started
3. The Memory Model: Stack, Heap, and Everything Between
4. Pointers, Arrays, and the Address-of Operator
5. Structs, Classes, and the Object Model
6. RAII: The Idea That Defines Modern C++
7. References, Copy, Move, and the Rule of Five
8. Smart Pointers: Ownership Without Leaks
9. Templates and Generic Programming
10. The STL: Containers, Iterators, Algorithms
11. Worked Mini-Example: A Word-Frequency Counter
12. Compilation and Linking in Depth
13. Undefined Behavior: The Dragons
14. Cheat Sheet and C vs C++ Comparison
15. Interview Questions & Answers
16. Further Learning


## What C and C++ Are, and Why They Still Matter

C is a thin, portable layer over the machine: you manage memory and the compiler trusts you completely. C++ keeps that zero-overhead philosophy but adds abstractions —

classes, templates, RAII, the STL — so you can build large systems without paying for features you don't use.

C (1972, Dennis Ritchie) is the *lingua franca* of systems programming: operating systems, embedded firmware, databases, language runtimes. C++ (1983, Bjarne Stroustrup) layers object-oriented and generic programming on top while preserving C's performance model. Both compile directly to native machine code — there is no virtual machine, no garbage collector, no interpreter standing between your code and the CPU.

> **Core principle**
> C++'s guiding rule is **"you don't pay for what you don't use"** (the zero-overhead principle). An abstraction like `std::vector` compiles to roughly the same code you'd write by hand with raw pointers — but it manages the memory for you.

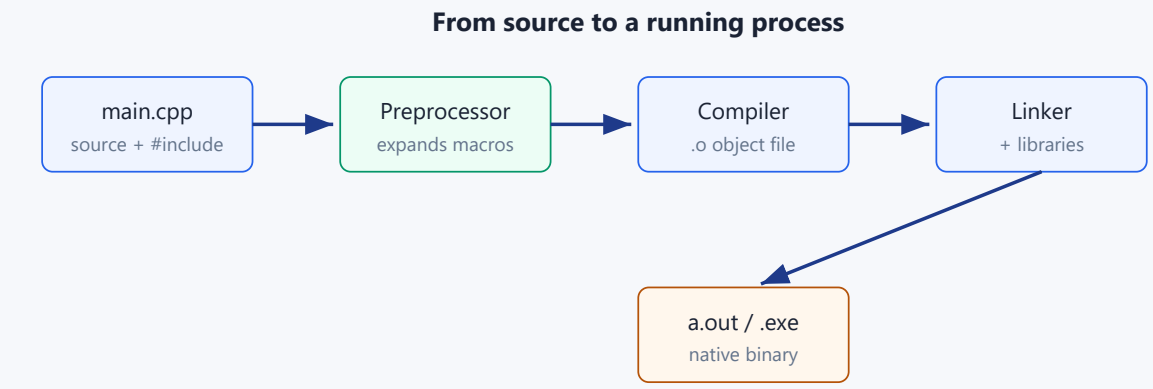



*Fig 1. The translation pipeline — preprocess, compile per translation unit, then link.*

# Setup and Getting Started

You need a compiler toolchain. GCC and Clang are the open-source standards; MSVC ships with Visual Studio on Windows.

BASH

```
# Install a toolchain
sudo apt install build-essential   # Debian/Ubuntu: gcc, g++, make
xcode-select --install             # macOS: clang
# Windows: install MSVC (Visual Studio) or MSYS2/MinGW

# Compile a single C++ file with warnings on and modern standard
g++ -std=c++23 -Wall -Wextra -O2 main.cpp -o app
./app

# Compile C
gcc -std=c17 -Wall -Wextra -O2 main.c -o app

# Debug build with sanitizers — catches memory bugs at runtime
g++ -std=c++23 -g -fsanitize=address,undefined main.cpp -o app
```

✦ **Always** compile with `-Wall -Wextra`. During development add `-fsanitize=address,undefined` — AddressSanitizer and UBSan catch use-after-free, buffer overflows, and undefined behavior that would otherwise be silent, intermittent crashes.

CPP

```cpp
// main.cpp — the canonical starting point
#include <iostream>   // std::cout, std::endl

int main() {
    std::cout << "Hello, systems programming\n";
    return 0;           // 0 = success to the OS
}
```

## The Memory Model: Stack, Heap, and Everything Between

Understanding where objects live is the single most important mental model in C and C++. A process has distinct memory regions, each with different lifetime and cost semantics.

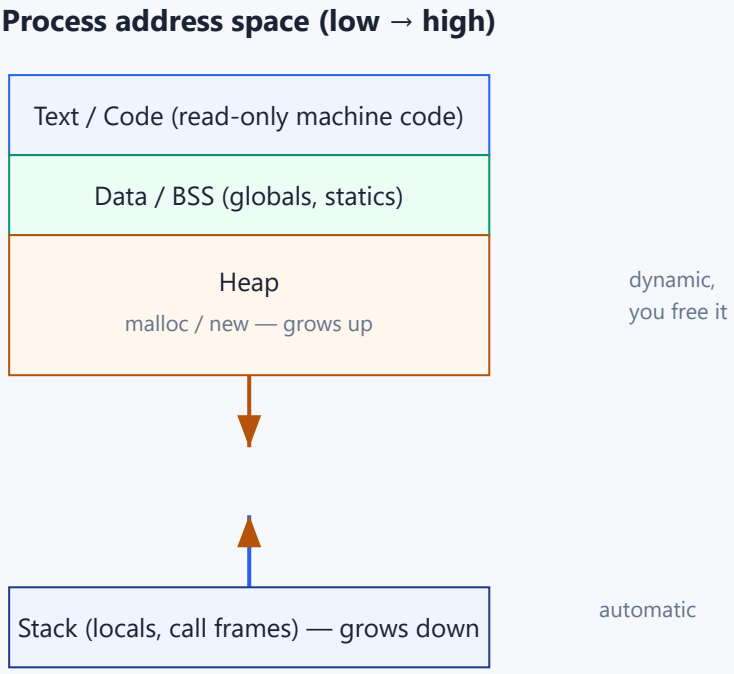



*Fig 2. Stack grows downward, heap upward; they share the free space between them.*

### STACK

Automatic lifetime. Allocation is a single register adjustment — essentially free. Freed automatically when the enclosing scope exits. Limited size (often 1-8 MB). Perfect for most local variables.

### HEAP

Dynamic lifetime that you control. Allocation (`malloc` / `new`) is comparatively expensive and may fail. You are responsible for freeing it — forget, and you leak; free twice, and you corrupt.

CPP

```cpp
void demo() {
    int x = 42;              // stack: freed when demo() returns
    int arr[100];            // stack: 400 bytes, freed automatically

    int* p = new int(7);     // heap: p is on the stack, *p is on the heap
    delete p;                // you MUST free it

    // Prefer this — RAII cleans up automatically (see below):
    auto up = std::make_unique<int>(7);
}   // up's destructor frees the heap int here
```

# Pointers, Arrays, and the Address-of Operator

A pointer is a variable whose value is a memory address. This is the heart of C. Arrays decay to pointers, which is both powerful and the source of countless bugs.

c

```c
int n = 10;
int* p = &n;        // p holds the address of n
printf("%d\n", *p);  // dereference: prints 10
*p = 20;             // writes through the pointer; n is now 20

int a[5] = {1, 2, 3, 4, 5};
int* q = a;          // array decays to pointer to its first element
printf("%d\n", *(q + 2));  // pointer arithmetic: prints a[2] == 3
printf("%d\n", q[2]);      // identical — a[i] is sugar for *(a + i)

// A pointer is NULL / nullptr when it points to nothing:
int* nothing = NULL;   // C
int* better  = nullptr; // C++ — type-safe null
```

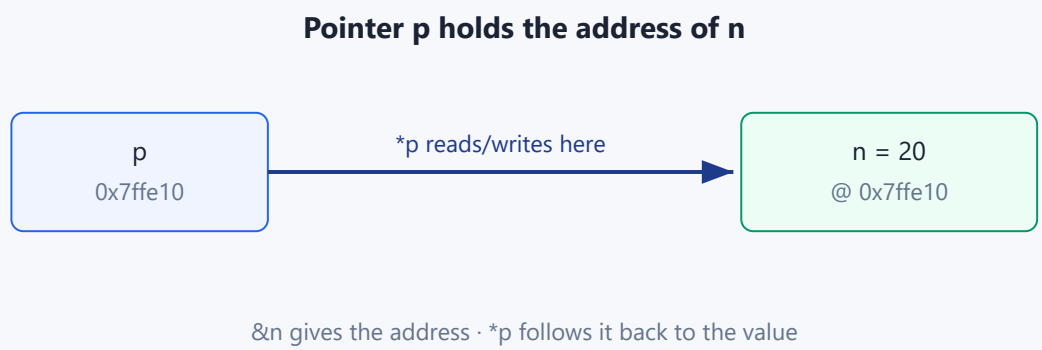



*Fig 3. The address-of (&) and dereference (*) operators are inverses.*

⚠ **Array decay**

When you pass an array to a function, you pass a pointer — the size is lost. `sizeof` inside the function returns the pointer size, not the array size. In C++ prefer `std::array` or `std::span`, which carry their length.

### const correctness

CPP

```cpp
const int* p1;        // pointer to const int: can't change *p1, can change p1
int* const p2 = &n;   // const pointer: can change *p2, can't change p2
const int* const p3 = &n; // both fixed
// Read right-to-left: p1 is a pointer to an int that is const.
```

## Structs, Classes, and the Object Model

A `struct` groups data. In C++ a `class` adds member functions, access control, constructors, and destructors. The only difference between `struct` and `class` in C++ is the default access level (public vs private).

CPP

```cpp
struct Point {        // C-style aggregate, still valid C++
    double x, y;
};

class Account {
    double balance_;              // private by default
public:
    explicit Account(double initial) : balance_(initial) {}  // constructor

    void deposit(double amt) { balance_ += amt; }
    double balance() const { return balance_; } // const member: promises no mutation

    ~Account() { /* destructor: runs when object dies */ }
};

Account a{100.0};
a.deposit(50);
std::cout << a.balance();  // 150
```

✦ **Member initializer lists**

Initialize members with `: balance_(initial)` rather than assigning inside the constructor body. It is required for `const` and reference members, and avoids a wasteful default-construct-then-assign.

## RAII: The Idea That Defines Modern C++

Resource Acquisition Is Initialization: tie every resource (memory, file handle, lock, socket) to the lifetime of an object. The constructor acquires; the destructor releases. Because destructors run deterministically when scope exits — even during exceptions — you cannot forget to clean up.

CPP

```cpp
#include <fstream>
#include <mutex>

void write_log(std::mutex& m, const std::string& msg) {
    std::lock_guard<std::mutex> lock(m);  // acquires lock in constructor
    std::ofstream file("log.txt", std::ios::app); // opens file (RAII)
    file << msg << '\n';
    // No manual unlock, no manual close:
    // ~ofstream closes the file, ~lock_guard releases the mutex,
    // AUTOMATICALLY, even if an exception is thrown above.
}
```




*Fig 4. RAII: cleanup is attached to scope exit, making leaks structurally impossible.*

# References, Copy, Move, and the Rule of Five

A reference is an alias for an existing object — it cannot be null and cannot be reseated. C++11 added *rvalue references* and move semantics, letting you transfer ownership of resources instead of copying them.

CPP

```cpp
void inc(int& r) { r++; }   // lvalue reference: modifies the caller's variable
int v = 5; inc(v);          // v == 6

void print(const std::string& s); // const ref: pass big objects without copying

// Move semantics: steal the guts of a temporary instead of deep-copying
std::vector<int> make() {
    std::vector<int> big(1'000'000, 0);
    return big;             // moved out, not copied (also NRVO)
}
std::vector<int> data = make();      // cheap
std::vector<int> copy = data;        // deep copy (expensive)
std::vector<int> moved = std::move(data); // steals data's buffer; data now empty
```

> **Rule of Five / Rule of Zero**
> If you define any of destructor, copy constructor, copy assignment, move constructor, or move assignment, you usually need all five. Better: the **Rule of Zero** — design classes so members (vectors, smart pointers) manage resources, and you write none of the five.

| Operation | Signature | When it runs |
|---|---|---|
| Copy constructor | `T(const T&)` | `T b = a;` |
| Copy assignment | `T& operator=(const T&)` | `b = a;` |
| Move constructor | `T(T&&)` | `T b = std::move(a);` |
| Move assignment | `T& operator=(T&&)` | `b = std::move(a);` |
| Destructor | `~T()` | scope exit / `delete` |

# Smart Pointers: Ownership Without Leaks

Raw `new` / `delete` is error-prone. Smart pointers encode *ownership* in the type and apply RAII to heap memory. In modern C++ you should almost never write a bare `delete`.

### UNIQUE_PTR

Sole owner, zero overhead. Move-only — cannot be copied. The default choice. `std::make_unique<T>(args)`.

### SHARED_PTR

Shared ownership via an atomic reference count. Frees when the last owner dies. Slight overhead. `std::make_shared<T>(args)`.

### WEAK_PTR

Non-owning observer of a `shared_ptr`. Breaks reference cycles. `lock()` to get a temporary `shared_ptr`.

CPP

```cpp
#include <memory>

struct Node { int val; std::shared_ptr<Node> next; };

auto u = std::make_unique<int>(42);  // sole owner
// auto u2 = u;                      // ERROR: can't copy a unique_ptr
auto u2 = std::move(u);              // OK: ownership transferred, u is now null

auto s1 = std::make_shared<Node>();  // refcount = 1
auto s2 = s1;                        // refcount = 2
// memory freed automatically when both s1 and s2 go out of scope

// Pass ownership vs. borrow:
void consume(std::unique_ptr<int> owned);   // takes ownership (by value)
void observe(const int& borrowed);          // just looks, no ownership
```

**Cyclic leak**

Two `shared_ptr`s pointing at each other never reach refcount 0 — a leak that smart pointers alone won't catch. Break the cycle by making one direction a `weak_ptr`.

# Templates and Generic Programming

Templates let you write code once that works for any type, resolved at compile time with no runtime cost. The STL is built entirely on them. C++20 *concepts* make template requirements explicit and error messages readable.

CPP

```cpp
#include <concepts>

// Function template
template <typename T>
T max_of(T a, T b) { return a > b ? a : b; }

// Class template
template <typename T>
class Box {
    T value_;
public:
    explicit Box(T v) : value_(std::move(v)) {}
    const T& get() const { return value_; }
};

// C++20 concept: constrain the type
template <std::integral T>        // only accepts integer types
T double_it(T x) { return x * 2; }

auto m = max_of(3, 7);            // T = int
auto s = max_of<std::string>("a", "b");
Box<double> b{3.14};
// double_it(3.5);                // compile error: double is not integral
```

✦ **Compile-time cost**

Templates are instantiated per type, which grows binary size and compile times. That is the price for runtime zero-overhead generics — a trade C++ deliberately makes.

# The STL: Containers, Iterators, Algorithms

The Standard Template Library is three interlocking parts: **containers** hold data, **iterators** traverse it, and **algorithms** operate on iterator ranges. This separation means one algorithm works across all containers.

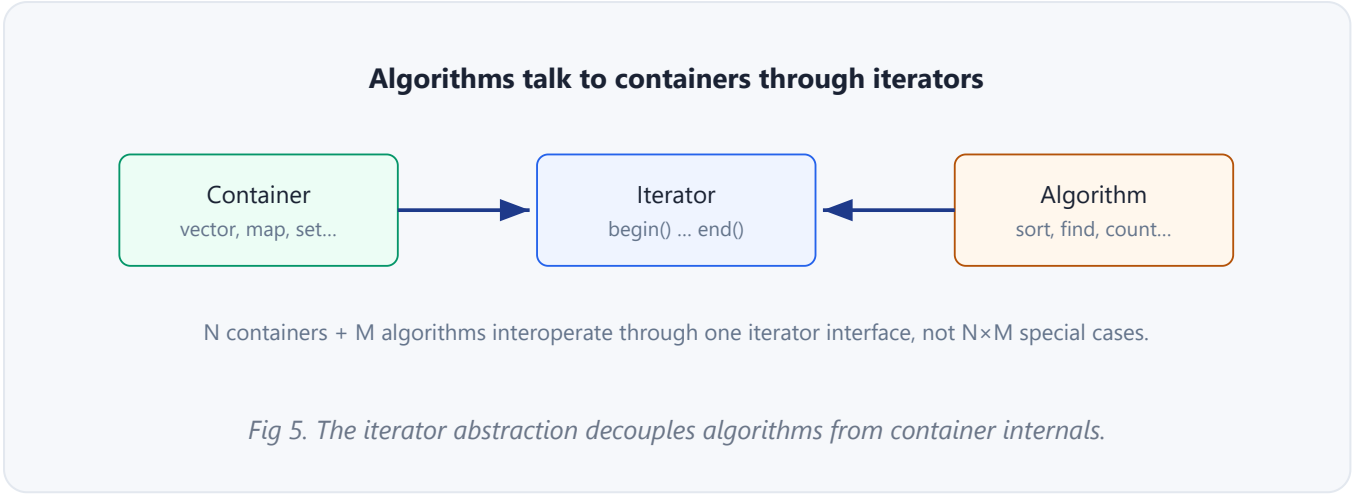



*Fig 5. The iterator abstraction decouples algorithms from container internals.*

CPP

```cpp
#include <vector>
#include <map>
#include <algorithm>
#include <numeric>

std::vector<int> v{5, 2, 8, 1, 9};
std::sort(v.begin(), v.end());               // 1 2 5 8 9
int total = std::accumulate(v.begin(), v.end(), 0);  // 25
auto it = std::find(v.begin(), v.end(), 8);  // iterator to 8

// Range-based for (C++11) and ranges (C++20)
for (int x : v) std::cout << x << ' ';

#include <ranges>
namespace rng = std::ranges;
auto evens = v | std::views::filter([](int x){ return x % 2 == 0; });

std::map<std::string, int> counts;
counts["apple"]++;        // inserts {"apple", 1} if absent, then increments
```

| Container | Underlying | Lookup | Insert | Ordered? |
|---|---|---|---|---|
| `vector` | dynamic array | O(1) index | O(1) amortized push_back | insertion order |
| `deque` | chunked array | O(1) index | O(1) both ends | insertion order |
| `list` | doubly linked | O(n) | O(1) with iterator | insertion order |
| `map` / `set` | red-black tree | O(log n) | O(log n) | sorted by key |
| `unordered_map` | hash table | O(1) average | O(1) average | no |

✦ **Default to vector**

Contiguous memory means cache-friendliness that usually beats "theoretically faster" node-based containers. Reach for `vector` first; profile before switching.

## Worked Mini-Example: A Word-Frequency Counter

This small program ties the pieces together — streams, strings, an unordered_map, and an algorithm — in idiomatic modern C++.

CPP

```cpp
#include <iostream>
#include <sstream>
#include <string>
#include <unordered_map>
#include <vector>
#include <algorithm>

int main() {
    const std::string text =
        "the quick brown fox the lazy dog the fox";

    std::unordered_map<std::string, int> freq;
    std::istringstream in(text);
    std::string word;
    while (in >> word) ++freq[word];   // operator[] default-constructs 0, then ++

    // Move pairs into a vector so we can sort by count (descending)
    std::vector<std::pair<std::string, int>> items(freq.begin(), freq.end());
    std::sort(items.begin(), items.end(),
              [](const auto& a, const auto& b){ return a.second > b.second; });

    for (const auto& [w, c] : items)   // structured bindings (C++17)
        std::cout << w << ": " << c << '\n';
    // the: 3, fox: 2, quick/brown/lazy/dog: 1
}
```

# Compilation and Linking in Depth

C and C++ use *separate compilation*: each `.cpp` (a translation unit) is compiled independently. Headers are copied in textually by the preprocessor. The linker stitches the resulting object files together.

CPP

```cpp
// math.h — declarations (the interface)
#ifndef MATH_H          // include guard: prevents double-inclusion
#define MATH_H
int add(int a, int b);  // declaration only
#endif

// math.cpp — definition (the implementation)
#include "math.h"
int add(int a, int b) { return a + b; }

// main.cpp — uses the declaration
#include "math.h"
int main() { return add(2, 3); }
```

BASH

```bash
# Compile each unit, then link
g++ -c math.cpp -o math.o     # -c = compile only, no link
g++ -c main.cpp -o main.o
g++ math.o main.o -o app      # link into executable
# "undefined reference to add" at this stage = linker can't find a definition
```

> **Declaration vs definition**
> A **declaration** says "this exists somewhere" (the compiler is happy). A **definition** provides the body (the linker needs exactly one — the One Definition Rule). The classic "undefined reference" error means you declared and used but never linked a definition.

# Undefined Behavior: The Dragons

Undefined behavior (UB) is the defining hazard of C and C++. The standard imposes no requirements — the program may crash, produce garbage, or appear to work until it doesn't. The compiler assumes UB never happens and optimizes accordingly, so a UB bug can delete your safety checks.

| Undefined behavior | Example | Defense |
|---|---|---|
| Dereferencing null/dangling | `*p` after `delete p` | smart pointers, set to nullptr |
| Buffer overflow | `a[10]` on `int a[10]` | `.at()`, `std::span`, ASan |
| Use-after-free | returning address of a local | RAII, don't return `&` local |
| Uninitialized read | `int x; use(x);` | always initialize |
| Signed overflow | `INT_MAX + 1` | use unsigned / check bounds |
| Double free | `delete p; delete p;` | unique_ptr |

CPP

```cpp
// CLASSIC BUG: returning a dangling reference
const std::string& broken() {
    std::string local = "oops";
    return local;             // local destroyed on return — UB to use result
}

// Dangling pointer from use-after-free
int* p = new int(5);
delete p;
std::cout << *p;              // UB: reading freed memory

// Off-by-one buffer overflow
int a[3];
for (int i = 0; i <= 3; ++i) a[i] = i;  // writes a[3] — out of bounds!
```

✗ **Never ignore UB**

"It works on my machine" is UB's favorite disguise. A read of uninitialized memory can pass tests for months, then corrupt data in production under a new compiler or optimization level. Run ASan/UBSan in CI.

## Cheat Sheet and C vs C++ Comparison

| Task | C | C++ (modern) |
|---|---|---|
| Heap alloc | `malloc/free` | `make_unique` / `make_shared` |
| Dynamic array | manual `realloc` | `std::vector` |
| String | `char*` + `strcpy` | `std::string` |
| Null | `NULL` | `nullptr` |
| Generic code | macros / `void*` | templates / concepts |
| Error handling | return codes / `errno` | exceptions / `std::expected` |
| I/O | `printf/scanf` | `std::cout` / `std::format` |
| Constants | `#define` | `constexpr` |

**write code** -> **-Wall -Wextra** -> **sanitizers** -> **profile** -> **optimize**

### Best practices

✓ **Do**

Prefer stack and RAII; use `std::vector` / `std::string` over raw arrays and `char*`; pass big objects by `const&`; mark functions and members `const` and `noexcept` where true; use `auto` for obvious types; initialize every variable; follow the Rule of Zero.

⚠ **Avoid**

Raw `new` / `delete`; C-style casts (use `static_cast`); owning raw pointers; macros for constants or functions; `using namespace std;` in headers; returning references/pointers to locals.

# Interview Questions & Answers

### What is the difference between the stack and the heap?

The stack holds automatic local variables and call frames; allocation is a cheap pointer bump and cleanup is automatic on scope exit, but size is limited. The heap is for dynamic-lifetime objects you allocate ( `new` / `malloc` ) and must free; allocation is slower and can fail or fragment. Prefer the stack and RAII.

### What is RAII and why is it central to C++?

Resource Acquisition Is Initialization binds a resource's lifetime to an object's lifetime: the constructor acquires, the destructor releases. Because destructors run deterministically on every scope exit — including during stack unwinding from an exception — resources (memory, locks, files) can't be leaked. It is the foundation of smart pointers, `lock_guard`, and `fstream`.

### unique_ptr vs shared_ptr — when do you use each?

`unique_ptr` models sole ownership with zero overhead and is move-only; it's the default. `shared_ptr` models shared ownership via an atomic reference count, with the cost of that counter and potential cycles. Use `shared_ptr` only when ownership is genuinely shared, and use `weak_ptr` to break cycles.

### What is a dangling pointer, and how do you prevent one?

A pointer that refers to memory which has been freed or whose object has gone out of scope. Dereferencing it is undefined behavior. Prevent it with RAII and smart pointers, by never returning the address of a local, and by not keeping raw pointers to objects you don't own. AddressSanitizer catches most at runtime.

### Explain the Rule of Five and the Rule of Zero.

If a class manages a resource and you define one of {destructor, copy ctor, copy assign, move ctor, move assign}, you likely need all five, because the compiler-generated versions won't handle the resource correctly. The Rule of Zero says: avoid this entirely by composing from members (vectors, smart pointers) that already manage their resources, so you write none of the five.

### What is move semantics and what problem does it solve?

Move semantics (C++11) let you transfer ownership of a resource (like a vector's heap buffer) from a temporary or soon-dead object instead of deep-copying it. An rvalue reference ( `T&&` ) binds to such objects; the move constructor steals the pointer and nulls the source. It turns an O(n) copy of a returned container into an O(1) pointer swap.

### Why pass by const reference?

Passing `const T&` avoids copying a potentially large object while guaranteeing the function won't modify it. It also lets the function accept temporaries. For small, cheap-to-copy types (int, pointers) pass by value instead.

### What is undefined behavior and why is it dangerous?

UB is a program construct for which the standard imposes no requirements — out-of-bounds access, signed overflow, use-after-free, uninitialized reads. The compiler assumes UB never occurs and may optimize away checks around it, so bugs are nondeterministic and can silently appear to work. There's no portable guarantee of a crash; always test with sanitizers.

### How does the compilation model work — declaration vs definition?

Each translation unit is compiled independently into an object file; headers are textually included. A declaration tells the compiler a name's type exists; a definition provides its body. The One Definition Rule requires exactly one definition across the program. The linker resolves symbols across object files — a missing definition yields "undefined reference" at link time, not compile time.

### vector vs list — which and why?

`vector` is a contiguous dynamic array: O(1) random access and excellent cache locality, O(1) amortized `push_back`, but O(n) middle insertion. `list` is a doubly linked list: O(1) splice/insert given an iterator but O(n) traversal and poor cache behavior. In practice `vector` wins almost always because cache locality dominates; prefer it unless you have measured a need for stable node addresses or frequent middle splicing.

### What does a virtual function do, and what is the vtable?

A `virtual` function enables runtime polymorphism: the call dispatches to the derived override based on the dynamic type. Implementations use a per-class vtable (array of function pointers); each polymorphic object stores a hidden vptr to its class's vtable. Always declare a `virtual` (or protected non-virtual) destructor in a base class you delete through a base pointer, or you get UB.

### What is the difference between a pointer and a reference?

A reference is an alias to an existing object: it must be initialized, can't be null, and can't be reseated. A pointer is a variable holding an address: it can be null, reassigned, and do arithmetic. Use references for "must refer to something" parameters; use pointers (ideally smart pointers) when you need nullability, reseating, or ownership transfer.

# Further Learning

- Read *A Tour of C++* (Stroustrup) for a fast, authoritative modern overview, and the **C++ Core Guidelines** for day-to-day rules.
- Keep cppreference.com open — it is the definitive, accurate reference for the standard library and language.
- Practice with sanitizers (ASan, UBSan, TSan) and Valgrind until catching UB becomes second nature.
- Learn CMake and a package manager (vcpkg or Conan) to build real multi-file projects and pull in dependencies.
- Study the newer standards incrementally: C++17 (structured bindings, `std::optional`), C++20 (concepts, ranges, modules), C++23 (`std::expected`, `std::print`).
- For interviews, implement classic data structures from scratch (dynamic array, hash map, LRU cache) in C++ to cement memory and ownership reasoning.